

where, $K(\mathbf{x}, \mathbf{y}) = \langle \Psi(\mathbf{x}), \Psi(\mathbf{y}) \rangle$. Following the gradient descent procedure for the minimization of this distance, we arrive at the following rule for the updates of the weights in the kernelized self-organizing map:

$$\mathbf{w}_i^{t+1} = \mathbf{w}_i^t - \alpha^t h^t \left(\frac{\partial}{\partial \mathbf{w}_i} K(\mathbf{w}, \mathbf{w}) \Big|_{\mathbf{w}_i^t} - 2 \frac{\partial}{\partial \mathbf{w}_i} K(\mathbf{x}, \mathbf{w}) \Big|_{\mathbf{w}_i^t} \right). \quad (9)$$

The completion of the training process results in a map $\kappa(\mathbf{x})$ from the high-dimensional space Ω to L

$$\kappa(\Psi(\mathbf{x})) : \mathbb{R}^N \rightarrow \Omega \rightarrow L, \quad (10)$$

which preserves the topological structure present in Ω . In the inference phase, similar to the original version of the algorithm, the weights are no longer updated; instead, the appropriate BMU for a data sample $\mathbf{x}$ is chosen. It was demonstrated that the kernelized version of the algorithm exhibited superior performance compared to the original version. This can be attributed to its ability to accurately identify the BMU in higher-dimensional spaces [51].

III. VARIATIONAL QUANTUM SELF-ORGANIZING MAPS

Equipped with the understanding of the classical self-organizing map in section II A and its kernelized counterpart in section II B, we are now ready to discuss the variational quantum self-organizing map (variational QSOM) for the unsupervised learning of the classical and quantum data.

Similar to the classical self-organizing map, variational QSOM is composed of two layers, an input layer and an output layer. The input layer feeds a data sample from the quantum dataset $\rho(\mathbf{x}) \in \mathcal{B}(\mathcal{H})$, whereas the output layer consists of a lattice of neurons $\{l_1, \dots, l_k\} \in L$ arranged in a particular topology, for example, a rectangular or hexagonal grid, as shown in Fig. 2. $\rho(\mathbf{x})$ corresponds to either the quantum data sample generated from a quantum computer or a classical data sample mapped to a quantum state via a quantum feature map, given by

$$\Phi : \mathbf{x} \rightarrow \rho(\mathbf{x}) = |U(\mathbf{x})\rangle\langle U^\dagger(\mathbf{x})|, \quad (11)$$

where $U(\mathbf{x})$ is a unitary composed of quantum gates. It should be noted that, to ensure the possibility of a quantum advantage on an unsupervised machine learning task it is necessary (but not sufficient) to establish that the map generated by $U(\mathbf{x})$ is sufficiently complex to be non-simulable on a classical computer. Note, $\mathcal{B}(\mathcal{H})$ refers to the set of all bounded operators on the Hilbert space of quantum states. To each lattice point in the grid is associated a parameterized quantum state $\rho(\theta_i) \in \mathcal{B}(\mathcal{H})$. The θ_i 's here, may correspond to the angles in the Pauli-rotation gates. In analogy with the classical

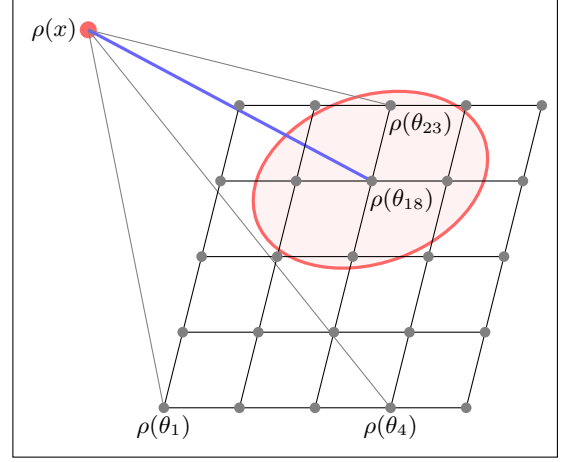


FIG. 2. A snapshot of the variational quantum self-organizing map during the training phase. Analogous to the classical self-organizing map, first the weights are randomly initialized and then the best matching unit (BMU) is found (here, l_{18}) by calculating the Hilbert-Shmidt norm between $\rho(\mathbf{x})$ and the quantum states corresponding to the assigned weights $\{\rho(\theta_1), \dots, \rho(\theta_k)\}$, see Eq. (13). The weights in the neighbourhood (here represented by a red circle) of the winning neuron are updated using the update rule as specified in Eq. (16).

SOM, the main objective of the variational QSOM is to learn the topology-preserving map $\kappa(\rho(\mathbf{x}))$ from a high dimensional continuous space $\mathcal{B}(\mathcal{H})$ to a low-dimensional lattice space L of k neurons

$$\kappa(\rho(\mathbf{x})) : \mathbb{R}^N \rightarrow \mathcal{B}(\mathcal{H}) \rightarrow L. \quad (12)$$

The learning of this mapping, with the help of quantum data, proceeds in three iterative steps as follows. A sample quantum state $\rho(\mathbf{x})$ is selected randomly from the given data. The second step comprises of finding the neuron l^* , i.e. *best matching unit* (BMU), which is closest to $\rho(\mathbf{x})$.

$$d[\rho(\mathbf{x}), \rho(\theta_{i^*}^t)] = \min_i \left\{ d[\rho(\mathbf{x}), \rho(\theta_i^t)] \right\}. \quad (13)$$

This is achieved by calculating the Hilbert-Shmidt inner product between $\rho(\mathbf{x})$ and the weight vector $\rho(\theta_i)$ associated with each neuron $l_i \in L$, as

$$K(\mathbf{x}, \theta_i) = \text{Tr}[\rho(\mathbf{x})\rho(\theta_i)] = |\langle 0|U^\dagger(\mathbf{x})U(\theta_i)|0\rangle|^2. \quad (14)$$

This is a crucial step in our algorithm; where the distance between the two states is estimated on a quantum computer by calculating the transition probability between the two states $\rho(\mathbf{x})$ and $\rho(\theta_i)$, as shown in Fig. 3. We also note that, for the unsupervised machine learning tasks, the embedding of the data sample and the weight vectors do not need to be identical. In theory, this algorithm can be used to calculate the overlap between two